



Figure 4.2.1. Sequential calls to the routine `Trapzd` incorporate the information from previous calls and evaluate the integrand only at those new points necessary to refine the grid. The bottom line shows the totality of function evaluations after the fourth call. The routine `qsimp`, by weighting the intermediate results, transforms the trapezoid rule into Simpson’s rule with essentially no additional overhead.

the latter equations. One example should give the idea, the formula with error term decreasing as $1/N^3$, which is closed on the right and open on the left:

$$\int_{x_0}^{x_{N-1}} f(x) dx = h \left[\frac{23}{12} f_1 + \frac{7}{12} f_2 + f_3 + f_4 + \cdots + f_{N-3} + \frac{13}{12} f_{N-2} + \frac{5}{12} f_{N-1} \right] + O\left(\frac{1}{N^3}\right) \quad (4.1.20)$$

CITED REFERENCES AND FURTHER READING:

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nrl.com/aands>, §25.4.[1]

Isaacson, E., and Keller, H.B. 1966, *Analysis of Numerical Methods*; reprinted 1994 (New York: Dover), §7.1.

4.2 Elementary Algorithms

Our starting point is equation (4.1.11), the extended trapezoidal rule. There are two facts about the trapezoidal rule that make it the starting point for a variety of algorithms. One fact is rather obvious, while the second is rather “deep.”

The obvious fact is that, for a fixed function $f(x)$ to be integrated between fixed limits a and b , one can double the number of intervals in the extended trapezoidal rule without losing the benefit of previous work. The coarsest implementation of the trapezoidal rule is to average the function at its endpoints a and b . The first stage of refinement is to add to this average the value of the function at the halfway point. The second stage of refinement is to add the values at the $1/4$ and $3/4$ points. And so on (see Figure 4.2.1).

As we will see, a number of elementary quadrature algorithms involve adding successive stages of refinement. It is convenient to encapsulate this feature in a `Quadrature` structure:

```
struct Quadrature{
```

quadrature.h

Abstract base class for elementary quadrature algorithms.

```
    Int n;                                Current level of refinement.
```

```
    virtual Doub next() = 0;
```

Returns the value of the integral at the nth stage of refinement. The function next() must be defined in the derived class.

```
};
```

Then the Trapzd structure is derived from this as follows:

```
template<class T>
```

quadrature.h

```
struct Trapzd : Quadrature {
```

Routine implementing the extended trapezoidal rule.

```
    Doub a,b,s;                            Limits of integration and current value of integral.
```

```
    T &func;
```

```
    Trapzd() {};
```

```
    Trapzd(T &funcc, const Doub aa, const Doub bb) :
```

```
        func(funcc), a(aa), b(bb) {n=0;}
```

The constructor takes as inputs func, the function or functor to be integrated between limits a and b, also input.

```
    Doub next() {
```

Returns the nth stage of refinement of the extended trapezoidal rule. On the first call (n=1), the routine returns the crudest estimate of $\int_a^b f(x)dx$. Subsequent calls set n=2,3,... and improve the accuracy by adding 2^{n-2} additional interior points.

```
        Doub x,tnm,sum,del;
```

```
        Int it,j;
```

```
        n++;
```

```
        if (n == 1) {
```

```
            return (s=0.5*(b-a)*(func(a)+func(b)));
```

```
        } else {
```

```
            for (it=1,j=1;j<n-1;j++) it <= 1;
```

```
            tnm=it;
```

```
            del=(b-a)/tnm;
```

This is the spacing of the points to be added.

```
            x=a+0.5*del;
```

```
            for (sum=0.0,j=0;j<it;j++,x+=del) sum += func(x);
```

```
            s=0.5*(s+(b-a)*sum/tnm);
```

This replaces s by its refined value.

```
            return s;
```

```
        }
```

```
    }
```

```
};
```

Note that Trapzd is templated on the whole struct and does not just contain a templated function. This is necessary because it retains a reference to the supplied function or functor as a member variable.

The Trapzd structure is a workhorse that can be harnessed in several ways. The simplest and crudest is to integrate a function by the extended trapezoidal rule where you know in advance (we can't imagine how!) the number of steps you want. If you want $2^M + 1$, you can accomplish this by the fragment

```
Ftor func;
```

Functor func here has no parameters.

```
Trapzd<Ftor> s(func,a,b);
```

```
for(j=1;j<=m+1;j++) val=s.next();
```

with the answer returned as val. Here Ftor is a functor containing the function to be integrated.

Much better, of course, is to refine the trapezoidal rule until some specified degree of accuracy has been achieved. A function for this is

quadrature.h

```

template<class T>
Doub qtrap(T &func, const Doub a, const Doub b, const Doub eps=1.0e-10) {
Returns the integral of the function or functor func from a to b. The constants EPS can be
set to the desired fractional accuracy and JMAX so that 2 to the power JMAX-1 is the maximum
allowed number of steps. Integration is performed by the trapezoidal rule.
    const Int JMAX=20;
    Doub s,olds=0.0;           Initial value of olds is arbitrary.
    Trapzd<T> t(func,a,b);
    for (Int j=0;j<JMAX;j++) {
        s=t.next();
        if (j > 5)             Avoid spurious early convergence.
            if (abs(s-olds) < eps*abs(olds) ||
                (s == 0.0 && olds == 0.0)) return s;
        olds=s;
    }
    throw("Too many steps in routine qtrap");
}

```

The optional argument `eps` sets the desired fractional accuracy. Unsophisticated as it is, routine `qtrap` is in fact a fairly robust way of doing integrals of functions that are not very smooth. Increased sophistication will usually translate into a higher-order method whose efficiency will be greater only for sufficiently smooth integrands. `qtrap` is the method of choice, e.g., for an integrand that is a function of a variable that is linearly interpolated between measured data points. Be sure that you do not require too stringent an `eps`, however: If `qtrap` takes too many steps in trying to achieve your required accuracy, accumulated roundoff errors may start increasing, and the routine may never converge. A value of 10^{-10} or even smaller is usually no problem in double precision when the convergence is moderately rapid, but not otherwise. (Of course, very few problems really require such precision.)

We come now to the “deep” fact about the extended trapezoidal rule, equation (4.1.11). It is this: The error of the approximation, which begins with a term of order $1/N^2$, is in fact *entirely even* when expressed in powers of $1/N$. This follows directly from the *Euler-Maclaurin summation formula*,

$$\begin{aligned}
 \int_{x_0}^{x_{N-1}} f(x)dx &= h \left[\frac{1}{2} f_0 + f_1 + f_2 + \cdots + f_{N-2} + \frac{1}{2} f_{N-1} \right] \\
 &\quad - \frac{B_2 h^2}{2!} (f'_{N-1} - f'_0) - \cdots - \frac{B_{2k} h^{2k}}{(2k)!} (f^{(2k-1)}_{N-1} - f^{(2k-1)}_0) - \cdots
 \end{aligned}
 \tag{4.2.1}$$

Here B_{2k} is a *Bernoulli number*, defined by the generating function

$$\frac{t}{e^t - 1} = \sum_{n=0}^{\infty} B_n \frac{t^n}{n!}
 \tag{4.2.2}$$

with the first few even values (odd values vanish except for $B_1 = -1/2$)

$$\begin{aligned}
 B_0 &= 1 & B_2 &= \frac{1}{6} & B_4 &= -\frac{1}{30} & B_6 &= \frac{1}{42} \\
 B_8 &= -\frac{1}{30} & B_{10} &= \frac{5}{66} & B_{12} &= -\frac{691}{2730}
 \end{aligned}
 \tag{4.2.3}$$

Equation (4.2.1) is not a convergent expansion, but rather only an asymptotic expansion whose error when truncated at any point is always less than twice the magnitude

of the first neglected term. The reason that it is not convergent is that the Bernoulli numbers become very large, e.g.,

$$B_{50} = \frac{495057205241079648212477525}{66}$$

The key point is that only even powers of h occur in the error series of (4.2.1). This fact is not, in general, shared by the higher-order quadrature rules in §4.1. For example, equation (4.1.12) has an error series beginning with $O(1/N^3)$, but continuing with all subsequent powers of N : $1/N^4$, $1/N^5$, etc.

Suppose we evaluate (4.1.11) with N steps, getting a result S_N , and then again with $2N$ steps, getting a result S_{2N} . (This is done by any two consecutive calls of `Trapzd`.) The leading error term in the second evaluation will be $1/4$ the size of the error in the first evaluation. Therefore the combination

$$S = \frac{4}{3}S_{2N} - \frac{1}{3}S_N \quad (4.2.4)$$

will cancel out the leading order error term. But there *is* no error term of order $1/N^3$, by (4.2.1). The surviving error is of order $1/N^4$, the same as Simpson's rule. In fact, it should not take long for you to see that (4.2.4) is *exactly* Simpson's rule (4.1.13), alternating $2/3$'s, $4/3$'s, and all. This is the preferred method for evaluating that rule, and we can write it as a routine exactly analogous to `qtrap` above:

```
template<class T>
Doub qsimp(T &func, const Doub a, const Doub b, const Doub eps=1.0e-10) {
Returns the integral of the function or functor func from a to b. The constants EPS can be
set to the desired fractional accuracy and JMAX so that 2 to the power JMAX-1 is the maximum
allowed number of steps. Integration is performed by Simpson's rule.
    const Int JMAX=20;
    Doub s,st,ost=0.0,os=0.0;
    Trapzd<T> t(func,a,b);
    for (Int j=0;j<JMAX;j++) {
        st=t.next();
        s=(4.0*st-ost)/3.0;           Compare equation (4.2.4), above.
        if (j > 5)                   Avoid spurious early convergence.
            if (abs(s-os) < eps*abs(os) ||
                (s == 0.0 && os == 0.0)) return s;
        os=s;
        ost=st;
    }
    throw("Too many steps in routine qsimp");
}
```

quadrature.h

The routine `qsimp` will in general be more efficient than `qtrap` (i.e., require fewer function evaluations) when the function to be integrated has a finite fourth derivative (i.e., a continuous third derivative). The combination of `qsimp` and its necessary workhorse `Trapzd` is a good one for light-duty work.

CITED REFERENCES AND FURTHER READING:

- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §3.1.
- Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §7.4.1 – §7.4.2.
- Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §5.3.